

Звіт з дослідницької практики

Дослідження ефективності авторського методу Parallel K-Means для кластеризації українських текстів

Порівняльний аналіз 20 послідовних серій експериментів від 5 жовтня 2026 року

Зміст

Вступ.....	2
1 Методика дослідження та авторський метод Parallel K-Means.....	3
1.1 Напрямок дослідження та постановка задачі.....	3
1.2 Принцип дії та реалізація авторського методу.....	4
1.3 Способи подання текстів.....	5
1.4 Алгоритми порівняння та справедливості експерименту.....	6
1.5 План дослідження та режими запуску.....	6
1.6 Методика вимірювання часу пам'яті та CPU.....	8
1.7 Критерії якості та перевірювані гіпотези.....	10
2 Результати експериментів та порівняння ефективності.....	12
2.1 Склад нової експериментальної серії.....	12
2.2 Порівняння якості та ресурсів для всіх подань текстів.....	13
2.3 Ефективність авторського методу порівняно з альтернативами.....	19
2.4 Початкові та повторні запуски і стабільність часу.....	21
2.5 Пам'ять і навантаження процесора на етапах виконання.....	23
2.6 Сукупна ефективність і практичний вибір.....	26
2.7 Перевірка гіпотез і зіставлення з науковими роботами.....	27
2.8 Власний внесок і напрями подальших досліджень.....	28
Висновки.....	30
Список використаних джерел.....	31
Додаток А Навантаження ядер для всіх комбінацій.....	32
Додаток Б Розподіли часу всіх комбінацій.....	37

Вступ

Автоматична кластеризація текстів дає змогу впорядковувати документи за змістом без наперед визначеного правила класифікації. Для українських текстів результат залежить від мовної обробки, способу числового подання та властивостей алгоритму групування. Ефективність такого інструмента визначається одночасно тематичною узгодженістю кластерів, тривалістю обчислень і потребою в ресурсах. Покращення одного показника не обов'язково означає покращення інших.

Центральним об'єктом порівняння є розроблений у межах роботи метод Parallel K-Means. Авторська реалізація організовує паралельне навчання кількох незалежно ініціалізованих моделей K-Means і обирає результат із найменшою внутрішньокластерною сумою квадратів відстаней. Практичне питання полягає в тому, чи виправдовує покращення розбиття витрати на додаткові старти, створення потоків і використання пам'яті.

Об'єкт дослідження — процес кластеризації природномовних документів. Предмет — вплив подання тексту, алгоритму, повторного виконання та організації багатостартового пошуку на якість і ресурсні витрати. Мета — оцінити ефективність авторського Parallel K-Means порівняно з вісьмома альтернативами та встановити умови його доцільного застосування.

Емпіричну основу становлять 20 послідовних серій в одному сеансі програми, проведених 5 жовтня 2026 року. Кожна серія охоплює п'ять векторизаторів і дев'ять алгоритмів; загалом проаналізовано 900 записів. Першу серію розглянуто окремо як початкову, решту 19 — як повторні виконання. Для кожної комбінації оцінено якість, типову швидкодію, розкид часу, відстежувану пам'ять і доступні показники CPU [1–3].

Завдання роботи охоплюють опис авторського методу, перевірку повноти експериментальної матриці, порівняння всіх комбінацій, аналіз початкового запуску, виявлення нетипових затримок і визначення меж ресурсних вимірювань. Окрему увагу приділено відмінності між прискоренням однакового обсягу роботи та витратами на розширений пошук кращого розв'язку.

Найвиразніше покращення відносно базового K-Means отримано для Sentence-BERT, проте додаткові старти збільшують час. Це визначає подальше завдання оптимізації авторського методу.

1 Методика дослідження та авторський метод Parallel K-Means

1.1 Напрямок дослідження та постановка задачі

Дослідження спрямоване на порівняння готових обчислювальних конфігурацій, кожна з яких поєднує спосіб подання тексту та кластеризатор. Документ спочатку перетворюється на послідовність токенів, далі — на числовий вектор. Алгоритм отримує матрицю цих векторів і повертає мітку групи для кожного документа. Еталонна тема використовується після кластеризації для оцінювання, а не як навчальна ознака.

Для методів із фіксованим числом груп задано $K = 5$, що відповідає п'ятьом темам демонстраційного корпусу. Це забезпечує змістовну основу порівняння, але не робить умови однаковими для алгоритмів, які самостійно визначають кількість кластерів. Для DBSCAN, HDBSCAN та Affinity Propagation додатково аналізуються кількість груп і шумові документи. Утворення одного кластера або віднесення всього корпусу до шуму розглядається як результат дослідження, а не вилучається з таблиць.

Вибір Parallel K-Means як основного методу зумовлений залежністю K-Means від початкових центрів. Один старт може завершитися невігідним локальним розв'язком. Перевірка кількох початкових станів розширює пошук, а їх одночасне виконання потенційно зменшує час відносно послідовного багатостартового варіанта. Досліджувана реалізація втілює цей принцип у спільному програмному конвеєрі [2].

Ефективність визначається за кількома незалежними ознаками. Тематична ефективність характеризує відповідність еталонним темам; обчислювальна — затримку та витрати пам'яті; експлуатаційна — поведінку під час першого й наступних запусків. Велике завантаження CPU саме по собі не є перевагою: воно має супроводжуватися корисним результатом у прийнятний час.

Дослідження поєднує аналіз реалізації та обчислювальний експеримент. Теоретична частина пояснює властивості відбору за inertia і межі паралелізму. Експериментальна частина перевіряє, які з цих властивостей проявляються на конкретних даних. Висновки стосуються зафіксованих параметрів; повна матриця доступних методів не означає перебору всіх можливих гіперпараметрів.

1.2 Принцип дії та реалізація авторського методу

Авторський метод реалізовано класом `ParallelKMeansClusterer` у модулі `clustering/parallel_kmeans.py`. На вхід подаються матриця ознак, число кластерів, спосіб ініціалізації, кількість незалежних стартів і кількість робочих потоків. Перевіряється допустимість K та додатність параметрів паралельного виконання. У досліджуваній конфігурації використано вісім стартів і чотири потоки.

Для кожного старту створюється окрема модель `KMeans` із `n_init = 1`. Зерна випадковості формуються послідовно від 42 до 49. `ThreadPoolExecutor` одночасно виконує не більше чотирьох кандидатів. Кожен кандидат повертає мітки документів і `inertia`. Після завершення всіх обчислень обираються мітки кандидата з мінімальним значенням цієї функції. Отже, паралелізм організовано на рівні незалежних повних запусків алгоритму.

Власний внесок полягає в реалізації та інтеграції керованого паралельного багатостартового пошуку, його параметризації й експериментальному оцінюванні для текстових подань. Використання відомого `K-Means` як внутрішнього компонента не зменшує інженерного змісту роботи, але визначає коректне формулювання новизни: розроблено авторську реалізацію методу пошуку, а не нову цільову функцію кластеризації.

Відбір гарантує лише мінімальну `inertia` серед фактично отриманих кандидатів. Він не гарантує глобального оптимуму, найбільшого `F1` або `ARI`. Мітки тем не беруть участі у виборі переможця, тому тематично кращий кандидат може бути відхилений, якщо його геометрична функція гірша. Це принципова властивість досліджуваного методу, яку потрібно враховувати під час інтерпретації результатів.

Загальний обсяг роботи зростає приблизно пропорційно кількості стартів, тоді як час залежить від одночасності, внутрішньої багатопотоковості бібліотек і накладних витрат. Для восьми однаково дорогих кандидатів та чотирьох потоків ідеалізований розклад містить дві хвили виконання. Реальні витрати на створення моделей, синхронізацію та копіювання даних можуть перекрити виграш на малому корпусі. Пряме підтвердження прискорення потребує порівняння тих самих восьми стартів у послідовному й паралельному режимах.

1.3 Способи подання текстів

Усі кластеризатори в межах одного подання отримують спільну матрицю ознак. Це дозволяє віднести різницю між ними до алгоритму та його налаштувань. Порівняння різних векторизаторів є порівнянням повних конфігурацій: вони відрізняються геометрією, розмірністю та джерелом навчальних знань.

Таблиця 1.1 — Реалізовані подання текстів

Подання	Спосіб отримання	Особливість витрат
TF-IDF	Частотні ваги; до 5000 ознак; щільна матриця після toarray	Побудова словника та матриці входить у вимірювання
Word2Vec	100 ознак; усереднення векторів слів; 20 епох	Навчання на поточному корпусі входить у час
FastText	100 ознак; підсловні компоненти; усереднення	Навчання та структури підслів потребують пам'яті
Doc2Vec	100 ознак; вектори документів; 20 епох	Вектори беруться для навчальних документів
Sentence-BERT	paraphrase-multilingual-MiniLM-L12-v2	Вимірюється encode; конструктор моделі поза таймером

Для Word2Vec, FastText і Doc2Vec сервіс встановлює `min_count = 1` та `workers = 1`; вікно контексту дорівнює 5. Отже, параметр «чотири потоки» належить Parallel K-Means, а не навчанню цих трьох моделей. Усереднення векторів слів втрачає порядок токенів. Навчання лише на 50 документах також обмежує кількість контекстів, доступних для формування змістовних подань [2].

Sentence-BERT використовує попередньо навчену модель, тому його результат спирається на знання, отримані поза досліджуванним корпусом. У поточному конвеєрі модель кодує рядки, зібрані з очищених токенів. Виграш такого подання не можна пояснити лише архітектурою: впливають і попереднє навчання, і конкретна модель, і попередня обробка.

Масштаб ознак має особливе значення для щільнісних методів. Стандартне TF-IDF нормується, а для усереднених векторів у коді немає окремого кроку L2-нормування. Той самий `eps = 0,5` тому задає різні за змістом околиці. Усі методи перевірено в заданих налаштуваннях, проте така перевірка не є окремо оптимізованим налаштуванням кожного простору.

1.4 Алгоритми порівняння та справедливість експерименту

Таблиця 1.2 — Роль алгоритмів у порівнянні

Алгоритм	Роль та зафіксовані умови
K-Means	База для оцінювання додаткових стартів; $K = 5$, <code>k-means++</code> , <code>n_init = auto</code> , <code>seed = 42</code>
Parallel K-Means	Авторський метод; $K = 5$, 8 стартів, 4 потоки, <code>seed = 42–49</code>
Ward	Агломеративне групування; 5 груп, <code>linkage = ward</code>
DBSCAN	Щільнісна альтернатива; <code>eps = 0,5</code> , <code>min_samples = 5</code>
MiniBatch	<code>MiniBatchKMeans</code> ; $K = 5$, <code>n_init = auto</code> , <code>seed = 42</code>
BIRCH	Ієрархія кластерних ознак; запитане $K = 5$
Spectral	<code>SpectralClustering</code> ; $K = 5$, <code>seed = 42</code> ; <code>affinity</code> явно не перевизначено
Affinity	<code>AffinityPropagation</code> ; число груп не задається; <code>seed = 42</code>
HDBSCAN	Пакет <code>hdbscan</code> ; <code>min_cluster_size = 5</code> , інші параметри типові

За `k-means++` параметр `n_init = auto` у сучасному `scikit-learn` означає один старт [4]. Таким чином, порівняння базового K-Means з авторським методом одночасно змінює обсяг пошуку й спосіб його виконання. Це доречне прикладне порівняння готових варіантів, але його часовий коефіцієнт не є чистим прискоренням від чотирьох потоків.

Для виділення ефекту паралелізму потрібна додаткова контрольна реалізація: вісім послідовних моделей із тими самими зернами, параметрами та правилом відбору. Налаштування `n_init = 8` саме по собі не гарантує тотожного набору початкових станів. Порівнювати потрібно фактичні кандидати та їхні `inertia`, а не лише однакове число стартів.

Внутрішні потоки `OpenMP` і `BLAS` утворюють ще один рівень паралелізму [5]. Чотири Python-потоки можуть запускати більше чотирьох обчислювальних потоків бібліотек. Тому кількість `workers` не можна ототожнювати з кількістю зайнятих ядер. У нових серіях цей внутрішній бюджет не зафіксовано окремо; його вплив розглядається як пояснення, що потребує перевірки.

1.5 План дослідження та режими запуску

Матриця дослідження містить $5 \times 9 = 45$ комбінацій. Вона повторена 20 разів без зміни конфігурації. Кожен файл містить один запис на комбінацію, а параметр `runs` дорівнює 1. Отже, статистична база для оцінювання

повторюваності формується між окремими серіями, а не з прихованих повторів усередині одного рядка. Для першої серії є 45 спостережень; для повторного режиму — 855.

Основним показником типової затримки обрано медіану 19 повторних запусків. Разом із нею наводяться середнє, вибіркове стандартне відхилення, квартилі та крайні значення там, де вони потрібні для пояснення поведінки. Стандартне відхилення обчислено з дільником 18. Велика різниця між середнім і медіаною розглядається як ознака асиметричного розподілу затримок, а не автоматично як помилка вимірювання.

Порівняння авторського й базового методів виконується окремо для кожного векторизатора. Різниці F1 та ARI подаються в абсолютних одиницях відповідних шкал. Відношення медіан часу характеризує ресурсну ціну розширеного пошуку. Для суми часу векторизації та кластеризації спочатку складаються показники в межах однієї серії, а вже потім обчислюється медіана; сума двох медіан загалом не є медіаною суми.

Векторизація виконується один раз на подання в кожній серії, а її показники повторено в дев'яти рядках кластеризаторів. Тому фактично доступні 100 вимірювань векторизації, з яких 95 належать повторному режиму. Дев'ять копій одного значення не трактуються як дев'ять незалежних вимірювань. Аналогічно під час обчислення сумарного часу серії векторизація враховується лише один раз.

Серії виконувалися послідовно в одному сеансі. Вони описують варіативність цього сеансу, проте не є незалежними випробуваннями на різних машинах або корпусах. Порядок методів незмінний, що залишає можливий вплив попередніх операцій. Тому статистичні характеристики використовуються як описові; формальні висновки про загальну значущість різниці чи довірчі інтервали для всіх середовищ не заявляються.

Холодний запуск і повторне виконання

Холодний запуск потрібно визначати через стан конкретних компонентів. Запуск нового процесу, перше завантаження моделі, перше звернення до дискового кешу та перший виклик обчислювального ядра є різними подіями. Перезапуск програми не обов'язково очищує файловий кеш операційної системи, а повторний виклик моделі може зберігати прогріті бібліотечні ресурси.

У дослідженні підтверджено 20 послідовних запусків в одному сеансі. Перша збережена серія позначається «початкова», а серії 2–20 — «повторні». Це спостережуване порівняння початкового та повторного виконання. Повністю холодний стан усіх кешів і бібліотек перед першою серією не зафіксовано, тому її не можна безумовно ототожнювати з контрольованим холодним експериментом.

Додатково сервіс перед кластеризацією перебирає K від 2 до 10 для рекомендації за Silhouette. Для кожного подання це дев'ять запусків K-Means і дев'ять оцінювань силуету. Таким чином, K-Means уже викликано до його профільованого результату навіть в початковій серії. Авторський метод також виконується після базового K-Means. Цей порядок обмежує інтерпретацію першого рядка кожного методу як «холодного».

Конструктор Sentence-BERT викликається до початку вимірювання `fit_transform`. Під час кожної серії створюється новий об'єкт векторизатора; постійне повторне використання одного об'єкта моделі не реалізовано. Водночас бібліотеки та файлові кеші можуть залишатися прогрітими. Тому повторний режим у цьому експерименті означає повторне проходження сервісу в одному процесі, а не обов'язково обслуговування запиту вже завантаженою моделлю.

Порівняння першого часу з медіаною наступних дає коефіцієнт початкового подорожчання. Це описовий індикатор, для якого є лише одне початкове спостереження. Якщо коефіцієнт менший за одиницю, це означає, що перший зафіксований виклик був швидшим за типові наступні; універсального правила про обов'язкову повільність початкового запуску дані не встановлюють.

1.6 Методика вимірювання часу пам'яті та CPU

Таймер `perf_counter` охоплює виклик `fit_transform` або `fit_predict`. Для векторизації вимірюються побудова подання та, для Gensim, навчання моделі всередині цього виклику. Для кластеризації вимірюється навчання й отримання міток. Створення більшості об'єктів кластеризаторів відбувається перед профілюванням; у Parallel K-Means моделі кандидатів і пул потоків створюються всередині `fit_predict` та входять у його час.

Профільований час не дорівнює повній тривалості дії користувача. За межами таймера залишаються завантаження даних, мовна обробка, конструктор

Sentence-BERT, підбір K, оцінювання якості, побудова графіків і експорт. Ці операції є частиною роботи програми, тому загальну швидкодію не можна оцінювати лише за найменшим `clustering_seconds`.

Пам'ять визначається через `tracemalloc`: на початку операції вмикається трасування, після завершення зчитується пік відстежуваних виділень і трасування припиняється. Значення `memory_kb` у файлах фактично виражене в КіБ, оскільки кількість байтів ділиться на 1024. Для зручності великі значення додатково переведено в МіБ.

Це не повна резидентна пам'ять процесу. Блоки, виділені до початку трасування, не входять у такий пік [7]. Також показник не є універсальним обліком усієї пам'яті нативних бібліотек або прискорювача. Низьке число для Sentence-BERT не означає, що модель повністю займає лише сотню КіБ: її створено раніше. Матриця ознак також уже існує до початку вимірювання кластеризації.

Піки окремих етапів не можна додавати як піковий обсяг RAM конвеєра. Частина об'єктів зберігається між етапами, частина звільняється, а моменти максимумів різні. Для повного профілю потрібні RSS або USS процесу, базовий рівень перед етапом, часовий ряд використання пам'яті та окремий облік пристрою виконання моделі. У цьому дослідженні порівнюються саме наявні піки відстежуваних виділень.

Вимірювання процесора та окремих ядер

Монітор CPU обчислює приріст процесорного часу процесу, додаючи `user` і `system`, та ділить його на тривалість моніторингу і кількість логічних процесорів. Після множення на 100 отримується нормований відсоток загальної доступної місткості CPU. Масиви по ядрах містять 12 позицій. За цієї нормалізації повне використання одного логічного процесора відповідає приблизно 8,33 %, а 100 % — еквіваленту дванадцяти повністю зайнятих логічних процесорів у вікні спостереження [2; 3].

Цей відсоток має інше вікно, ніж таймер алгоритму. Монітор запускається до `tracemalloc`, а зупиняється після нього. Фоновий потік опитування засинає на 0,1 с; `stop` очікує його завершення через `join`. Очікування входить у знаменник CPU, але не входить у `clustering_seconds`. Тому множення збереженого CPU на час кластеризації не відновлює достовірно процесорні секунди алгоритму.

Показники `cpu_per_core_percent` отримуються через системний `psutil.cpu_percent`, а не через атрибуцію потоків конкретного процесу. Вони включають фонові застосунки. Програма не зберігає часових міток окремих вибірок, кількості вибірок у результатах або відповідності робочих потоків ядрам. Ці масиви описують системну обстановку, але не дозволяють розподілити роботу Parallel K-Means між дванадцятьма ядрами.

Є додаткова похибка першої вибірки: ініціалізаційний виклик `psutil` зроблено в головному потоці, тоді як вибірки збирає інший потік. Документація застерігає щодо першого неблокувального вимірювання; у версіях із незалежною історією викликів потоків таке попереднє опитування не встановлює потрібну базу для нового потоку [6]. Це обмежує придатність навіть ненульових коротких вибірок.

У 898 із 900 кластеризацій тривалість менша за інтервал опитування 100 мс. Високі значення CPU можуть охоплювати активність бібліотечних потоків поза коротким `fit_predict`; нульові значення по ядрах також не доводять відсутність обчислень. Тому ресурсний висновок спирається насамперед на час і коректно визначену пам'ять. CPU аналізується окремо з урахуванням цих обмежень.

1.7 Критерії якості та перевірювані гіпотези

Silhouette описує співвідношення внутрішньогрупових і міжгрупових відстаней. Davies–Bouldin характеризує відносне розсіювання груп; менші значення бажані. Calinski–Harabasz зіставляє міжгрупову та внутрішньогрупову варіативність. Ці критерії оцінюють геометрію конкретного простору, тому їх не слід використовувати як єдине свідчення тематичної якості.

F1 macro обчислюється після взаємно однозначного зіставлення кластерів із темами за максимальною кількістю збігів. Коли кластерів більше за теми, зайві групи отримують спеціальну неспівставлену мітку; вона впливає на усереднення. Тому F1 для Affinity Propagation із вісьмома або тринадцятьма групами потребує додаткового пояснення. ARI порівнює пари об'єктів із поправкою на випадковість; NMI та V-measure характеризують інформаційну узгодженість. Рівність двох останніх у цьому експорті не є двома незалежними підтвердженнями.

Шумові документи виключаються із зовнішніх метрик. За відсутності нешумових об'єктів програма записує нулі. Для внутрішніх метрик значення -1

може бути службовим маркером неможливості оцінювання. Воно позначене як «н/о» у таблицях для вироджених розбиттів. Невеликий від'ємний силует HDBSCAN на TF-IDF збережено як справжній обчислений результат; він не замінюється маркером.

Гіпотеза Н1: авторський багатостартовий метод покращує тематичну якість відносно базового K-Means принаймні для частини подань. Гіпотеза Н2: за однакового обсягу пошуку паралельне виконання скорочує час порівняно з послідовними вісьмома стартами. Н1 перевіряється наявними метриками; Н2 вимагає окремої рівноцінної бази порівняння.

Гіпотеза Н3: початковий запуск має більші витрати для компонентів із відкладеною ініціалізацією. Гіпотеза Н4: характер навантаження CPU та пам'яті залежить від етапу конвеєра й подання. Для Н3 аналізується співвідношення початкової серії з повторними; для Н4 використовуються доступні етапні показники та аудит вимірювача. Жодна гіпотеза не вважається підтвердженою лише тому, що вона була метою розробки.

2 Результати експериментів та порівняння ефективності

2.1 Склад нової експериментальної серії

Новий масив охоплює 20 каталогів від `experiment_20261005_093016` до `experiment_20261005_093602`. У кожному перевірено наявність усіх 45 унікальних пар «векторизатор — алгоритм». Пропущених комбінацій, дублювання пар усередині серії та записів із полем `error` не виявлено. Усі 900 записів містять результати для 50 документів. Конфігурації серій збігаються [1].

Корпус `demo_texts.csv` містить п'ять тематичних класів по десять документів. Стоп-слова та лематизацію ввімкнено в конфігурації. Це зафіксований режим запуску; окремі журнали отриманих лем до архіву не включено. Тому вплив лематизації не виділяється як самостійний експериментальний чинник. На відміну від попереднього дослідження, тепер для всіх п'яти подань використано однакове запитане $K = 5$.

У таблиці наведено часові позначки початку каталогів. Тривалість між ними та `created_at` у `manifest` є лише приблизною часовою рамкою проходження сервісу з точністю до секунди, а не окремим високоточним вимірюванням повного запуску.

Таблиця 2.1 — Послідовність експериментів 5 жовтня 2026 року

Серії	Початки серій за назвами каталогів	Записи
1–5	09:30:16; 09:30:55; 09:31:12; 09:31:28; 09:31:46	225
6–10	09:32:05; 09:32:21; 09:32:37; 09:32:54; 09:33:12	225
11–15	09:33:27; 09:33:46; 09:34:03; 09:34:20; 09:34:37	225
16–20	09:34:53; 09:35:11; 09:35:27; 09:35:44; 09:36:02	225

Перевірка восьми показників якості — `Silhouette`, `Davies–Bouldin`, `Calinski–Harabasz`, `F1 macro`, `F1 micro`, `ARI`, `NMI` та `V-measure` — показала однакові значення для кожної пари в усіх 20 серіях із точністю експорту. Це дозволяє подавати одну таблицю якості без двадцятикратного дублювання. Повтори характеризують коливання ресурсів за сталого результату, а не двадцять незалежних підтверджень узагальнювальної здатності.

Далі «медіана», «середнє» і «розкид» ресурсів стосуються серій 2–20, якщо прямо не вказано інше. Час кластеризації наведено в мілісекундах, пам'ять — у КіБ відстежуваних виділень. Позначення `K/шум` показує фактичне число груп і кількість шумових документів.

2.2 Порівняння якості та ресурсів для всіх подань текстів

	TF-IDF	Word2Vec	FastText	Doc2Vec	Sentence-BERT
K-Means	0.272	0.309	0.343	0.306	0.642
Parallel K-Means	0.258	0.322	0.286	0.328	0.702
Ward	0.404	0.417	0.365	0.305	0.838
DBSCAN	0.000	0.067	0.067	0.067	0.000
MiniBatch	0.232	0.286	0.258	0.276	0.756
BIRCH	0.404	0.067	0.067	0.067	0.838
Spectral	0.327	0.334	0.353	0.338	0.638
Affinity	0.270	0.310	0.329	0.257	0.557
HDBSCAN	0.152	0.000	0.000	0.000	0.000

Рисунок 2.1 — F1 macro для всіх 45 комбінацій; значення незмінні в 20 серіях

Найвищу тематичну узгодженість отримано для Sentence-BERT із Ward та BIRCH: F1 macro = 0,8385, ARI = 0,6716. Авторський Parallel K-Means із цим поданням має F1 macro = 0,7024 та ARI = 0,6072. Він перевищує базовий K-Means за обома критеріями, проте не є загальним лідером. MiniBatch має вищий F1, але нижчий ARI, що показує залежність ранжування від обраного показника.

Для TF-IDF, Word2Vec, FastText і Doc2Vec значення ARI переважно близькі до нуля. Навіть помірне збільшення F1 у цих просторах не означає сильного відтворення тематичної структури. Тому подання тексту в цьому досліді впливає на практичну придатність результату сильніше, ніж сама наявність кількох стартів K-Means.

Нульові клітинки DBSCAN та HDBSCAN у частині подань відповідають віднесенню всіх документів до шуму. Значення близько 0,0667 для DBSCAN і BIRCH на усереднених векторах відповідає одній групі з усіма документами. Це різні типи невдалого розбиття, які не слід об'єднувати в одну категорію «низька точність».

Карту зовнішньої якості доповнюють наступні таблиці кількості кластерів, Silhouette та ресурсів.

Результати для TF-IDF

Таблиця 2.2 — TF-IDF: якість і медіани ресурсів повторних запусків

Метод	К/ шум	Sil.	F1	ARI	Час, мс	Пам., КіБ	CPU, %
K-Means	5/0	0,0046	0,2718	0,0225	2,730	303,5	91,6
Parallel K-Means	5/0	0,0101	0,2576	0,0019	21,008	1016,9	98,2
Ward	5/0	0,0123	0,4041	0,0321	0,843	22,9	47,6
DBSCAN	0/50	н/о	0,0000	0,0000	0,988	14,7	91,5
MiniBatch	5/0	0,0012	0,2320	0,0042	6,546	272,8	91,8
BIRCH	5/0	0,0123	0,4041	0,0321	2,187	396,8	92,2
Spectral	5/0	0,0121	0,3273	0,0202	12,603	93,0	92,8
Affinity	13/0	0,0181	0,2698	0,0100	8,439	142,2	36,8
HDBSCAN	2/12	-0,0006	0,1519	-0,0105	2,086	27,3	1,5

На TF-IDF авторський метод збільшує Silhouette з 0,0046 до 0,0101, проте F1 macro зменшується з 0,2718 до 0,2576, а ARI — з 0,0225 до 0,0019. Вибір мінімальної inertia дав геометрично інший результат без покращення тематичної відповідності. Отже, у цьому поданні додаткові старти не виправдалися за зовнішніми критеріями.

Ward і BIRCH мають однакові збережені метрики якості, включно з F1 macro = 0,4041. Рівність агрегованих показників сама по собі не є перевіркою тотожності міток кожного документа. Медіана Ward становить 0,843 мс; Parallel K-Means — 21,008 мс. За цього корпусу Ward є сильнішою альтернативою за якістю та часом.

Affinity утворює 13 груп, тому його F1 = 0,2698 не описує те саме п'ятикластерне завдання. HDBSCAN залишає 38 нешумових документів і дві групи: охоплення становить 76 %. Його зовнішні оцінки стосуються саме цих 38 об'єктів, тоді як внутрішнє оцінювання може враховувати шум як окрему мітку. DBSCAN відносить усі 50 документів до шуму.

TF-IDF є найдешевшим за часом формування подання, але це не робить його найкращим за змістом кластерів. Для авторського методу цей результат особливо важливий: прискорення або збільшення кількості стартів не компенсує автоматично недостатню семантичну виразність ознак.

Результати для Word2Vec

Таблиця 2.3 — Word2Vec: якість і медіани ресурсів повторних запусків

Метод	К/ шум	Sil.	F1	ARI	Час, мс	Пам., KiB	CPU, %
K-Means	5/0	0,0014	0,3091	0,0050	2,941	78,3	91,8
Parallel K-Means	5/0	0,0123	0,3220	0,0227	22,968	294,8	98,3
Ward	5/0	0,0255	0,4174	0,0289	0,779	51,9	51,1
DBSCAN	1/0	н/о	0,0667	0,0000	1,131	54,8	91,3
MiniBatch	5/0	0,0017	0,2859	0,0126	4,781	79,1	92,4
BIRCH	1/0	н/о	0,0667	0,0000	1,415	53,9	91,9
Spectral	5/0	0,0215	0,3336	-0,0232	13,209	132,3	93,1
Affinity	4/0	0,0098	0,3100	0,0335	4,247	143,4	41,7
HDBSCAN	0/50	н/о	0,0000	0,0000	1,760	60,3	1,4

Parallel K-Means підвищує F1 макро з 0,3091 до 0,3220, ARI — з 0,0050 до 0,0227, Silhouette — з 0,0014 до 0,0123. Напряв зміни трьох критеріїв узгоджений, але абсолютні ARI залишаються низькими. Покращення є локальним результатом цієї конфігурації, а не свідченням високої тематичної якості загалом.

Ward дає найбільший F1 серед розглянутих алгоритмів цього подання — 0,4174 — за медіани часу близько 0,779 мс. Авторський метод потребує 22,968 мс за медіаною. У двох серіях його час перевищує 300 мс; ці спостереження суттєво підвищують середнє й докладно розглядаються в підрозділі 2.4.

BIRCH і DBSCAN утворюють по одній групі, а HDBSCAN залишає весь корпус у шумі. Вектори слів усереднюються, і в коді немає окремого вирівнювання їхніх норм. Тому задані пороги та стандартні налаштування щільнісних моделей не забезпечили тематичного поділу.

Час повторної векторизації становить приблизно 24,729 мс за медіаною. Це навчання Word2Vec на поточних текстах разом із усередненням векторів, а не звернення до готової великої моделі. Практична інтерпретація результату має враховувати малий обсяг навчальних контекстів і те, що зовнішнє попереднє навчання тут не використовувалося.

Результати для FastText

Таблиця 2.4 — FastText: якість і медіани ресурсів повторних запусків

Метод	К/ шум	Sil.	F1	ARI	Час, мс	Пам., KiB	CPU, %
K-Means	5/0	-0,0216	0,3429	0,0441	2,894	78,3	92,2
Parallel K-Means	5/0	0,0223	0,2864	-0,0110	23,133	287,2	98,0
Ward	5/0	0,0077	0,3651	0,0683	0,783	51,9	47,5
DBSCAN	1/0	н/о	0,0667	0,0000	1,130	54,8	91,3
MiniBatch	5/0	0,0385	0,2578	-0,0045	4,422	79,1	92,3
BIRCH	1/0	н/о	0,0667	0,0000	1,412	53,9	92,0
Spectral	5/0	0,0084	0,3527	0,0581	12,535	132,2	92,9
Affinity	9/0	0,0085	0,3289	0,0556	4,279	143,4	35,8
HDBSCAN	0/50	н/о	0,0000	0,0000	1,700	60,3	1,5

На FastText авторський метод збільшує Silhouette з $-0,0216$ до $0,0223$, але F1 масо зменшується з $0,3429$ до $0,2864$, а ARI — з $0,0441$ до $-0,0110$. Це найвиразніше погіршення зовнішньої якості Parallel K-Means відносно базового варіанта серед п'яти подань. Результат показує, що оптимізація евклідової компактності та відтворення тематичних класів можуть розходитися.

Найбільший F1 у цьому поданні отримано для Ward — $0,3651$. MiniBatch має більший Silhouette, ніж Parallel K-Means, але нижчий F1. Отже, ранжування за внутрішньою метрикою не можна переносити на тематичну придатність. Для вибору алгоритму потрібне зіставлення обох груп критеріїв.

Основна ресурсна особливість FastText проявляється на етапі векторизації. Медіана часу становить $531,652$ мс, а пік відстежуваних виділень — близько $763,492$ MiB. Кластеризація додає до цього значно меншу затримку. Тому висновок про економність конфігурації лише за `clustering_memory_kb` був би помилковим.

Підслівні структури можуть бути корисними для морфологічно багаті мови, проте їхня наявність не гарантує переваги за малого навчального корпусу. У цих умовах витрати FastText є значними, а якість авторського методу поступається Sentence-BERT. Це мотивує перевірку попередньо навчених векторів і обмеження параметрів підслівної моделі як окремих майбутніх експериментів.

Результати для Doc2Vec

Таблиця 2.5 — Doc2Vec: якість і медіани ресурсів повторних запусків

Метод	К/ шум	Sil.	F1	ARI	Час, мс	Пам., KiB	CPU, %
K-Means	5/0	0,0101	0,3059	-0,0016	2,802	78,3	91,4
Parallel K-Means	5/0	0,0096	0,3277	-0,0175	22,986	278,1	98,3
Ward	5/0	0,0215	0,3051	-0,0241	0,789	51,9	50,8
DBSCAN	1/0	н/о	0,0667	0,0000	1,412	54,8	91,4
MiniBatch	5/0	0,0031	0,2761	-0,0303	5,288	79,0	92,4
BIRCH	1/0	н/о	0,0667	0,0000	1,420	53,9	91,8
Spectral	5/0	0,0118	0,3384	0,0051	13,548	132,2	93,0
Affinity	9/0	0,0107	0,2573	-0,0063	5,129	143,4	39,4
HDBSCAN	0/50	н/о	0,0000	0,0000	1,847	60,2	1,6

Для Doc2Vec Parallel K-Means підвищує F1 макро з 0,3059 до 0,3277, проте ARI зменшується з -0,0016 до -0,0175. Silhouette також дещо зменшується. Таким чином, формулювання «якість покращилася» без уточнення конкретної метрики було б неповним. Перестановка документів між групами вигідна для оптимального зіставлення класів за F1, але не для парної узгодженості ARI.

Найбільший F1 у цьому поданні має Spectral — 0,3384, його ARI становить лише 0,0051. Значення всіх основних варіантів свідчать про слабку відповідність тематичній структурі. Перевага одного з них за кількома сотими F1 не змінює загальної оцінки цього подання на досліджуваному корпусі.

Медіана часу авторського методу — 22,986 мс проти 2,802 мс базового K-Means. Пік відстежуваних виділень збільшується приблизно у 3,55 раза. Для конфігурації з розбіжними критеріями якості така ресурсна ціна потребує обережного обґрунтування й не дає підстав рекомендувати метод без додаткової перевірки.

Doc2Vec навчається на тих самих документах, для яких потім беруться вектори model.dv. Експеримент оцінює групування навчального корпусу, а не здатність ефективно векторизувати нові документи через окремий inference. Це різні сценарії; наведений час і метрики не можна без перевірки переносити на потік раніше невідомих текстів.

Результати для Sentence-BERT

Таблиця 2.6 — Sentence-BERT: якість і медіани ресурсів повторних запусків

Метод	К/ шум	Sil.	F1	ARI	Час, мс	Пам., КіБ	CPU, %
K-Means	5/0	0,0416	0,6423	0,3664	3,373	261,7	91,7
Parallel K-Means	5/0	0,0490	0,7024	0,6072	25,111	1066,4	98,2
Ward	5/0	0,0899	0,8385	0,6716	0,886	162,9	51,5
DBSCAN	0/50	н/о	0,0000	0,0000	1,494	14,7	91,1
MiniBatch	5/0	0,0793	0,7563	0,5029	5,363	261,1	92,4
BIRCH	5/0	0,0899	0,8385	0,6716	2,405	406,7	92,1
Spectral	5/0	0,0591	0,6376	0,5694	13,407	243,2	92,9
Affinity	8/0	0,0600	0,5569	0,4780	4,807	354,0	37,4
HDBSCAN	0/50	н/о	0,0000	0,0000	2,499	180,0	1,5

Sentence-BERT дає найсильніші результати для авторського методу: F1 macro = 0,7024, ARI = 0,6072. Порівняно з базовим K-Means приріст становить відповідно 0,0601 та 0,2408. Обидві зміни позитивні, тому саме це подання найкраще ілюструє практичну користь багатостартового пошуку в проведеному дослідженні.

Водночас Ward і BIRCH досягають F1 macro = 0,8385 та ARI = 0,6716. MiniBatch має F1 = 0,7563, але ARI = 0,5029: він кращий за авторський метод за першим критерієм і гірший за другим. Spectral має нижчі за Parallel K-Means F1 та ARI, хоча виконується швидше. Вибір між цими альтернативами залежить від вимог до структури помилок і затримки.

Медіана часу Parallel K-Means становить 25,111 мс, базового K-Means — 3,373 мс, Ward — 0,886 мс. Пік відстежуваних виділень авторського методу дорівнює 1066,382 КіБ. Ці числа характеризують лише етап кластеризації; розмір попередньо завантаженої трансформерної моделі в них не міститься.

DBSCAN та HDBSCAN знову відносять усі документи до шуму. Це показує, що семантично змістовне подання саме по собі не усуває потреби в доборі щільнісних параметрів. Affinity утворює вісім груп і має F1 = 0,5569; його результат слід розглядати як інший рівень деталізації, а не безпосередній п'ятикластерний еквівалент.

2.3 Ефективність авторського методу порівняно з альтернативами

Таблиця 2.7 — Приріст якості та ресурсна ціна авторського методу

Подання	$\Delta F1$	ΔARI	Час Р/К	Пам'ять Р/К
TF-IDF	-0,0142	-0,0206	7,70	3,35
Word2Vec	0,0130	0,0177	7,81	3,77
FastText	-0,0566	-0,0552	7,99	3,67
Doc2Vec	0,0218	-0,0158	8,20	3,55
Sentence-BERT	0,0601	0,2408	7,44	4,08

Позначення Р/К означає відношення медіан Parallel K-Means до K-Means у серіях 2–20. Воно характеризує дорожчий або дешевший спосіб отримати кінцевий результат у наявних конфігураціях. Оскільки авторський метод перевіряє вісім кандидатів, а базовий — один, цей коефіцієнт не є оцінкою прискорення паралельного виконання однакової роботи.

F1 покращився для трьох із п'яти подань: Word2Vec, Doc2Vec і Sentence-BERT. ARI покращився для двох: Word2Vec та Sentence-BERT. Для TF-IDF і FastText обидва зовнішні показники зменшилися. Найбільший узгоджений позитивний ефект отримано на Sentence-BERT; найвиразніший негативний — на FastText. Таким чином, користь авторського методу залежить від сформованого простору ознак.

Типовий час зростає у 7,44–8,20 рази, а відстежувана пам'ять — у 3,35–4,08 рази. На невеликій матриці витрати запуску кількох кандидатів не компенсуються паралелізмом відносно одноразового K-Means. Проте з цих даних не випливає, що одночасні вісім стартів повільніші за ті самі вісім послідовних: останню конфігурацію не виміряно.

Порівняння має два практичні прочитання. Якщо потрібен найдешевший один результат, авторська конфігурація є дорожчою. Якщо потрібен пошук серед восьми початкових станів, вона реалізує керований механізм такого пошуку, але його перевагу в часі потрібно встановити рівноцінним контролем. Саме розділення цих питань дає об'єктивну оцінку власної розробки.

Порівняння з рештою алгоритмів

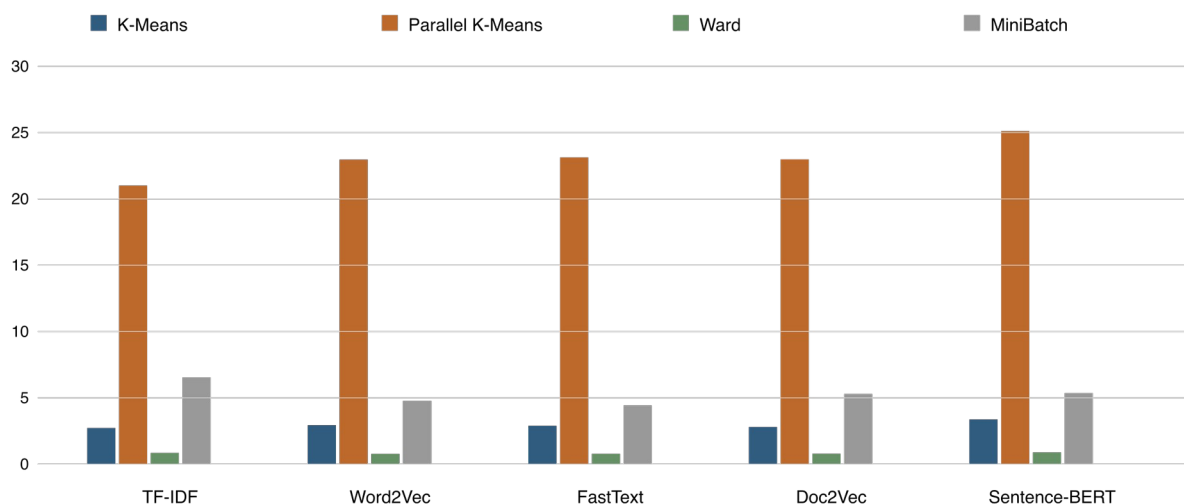


Рисунок 2.2 — Медіани часу кластеризації у мілісекундах для чотирьох основних альтернатив

На рисунку показано типову затримку після початкової серії. Ward виконується швидше за авторський метод у всіх п'яти поданнях. Parallel K-Means має найбільшу медіану `clustering_seconds` серед дев'яти алгоритмів для кожного подання. Такий результат узгоджується з більшим числом кандидатів, але не визначає поведінку на великих корпусах.

За F1 авторський метод посідає шосте місце на TF-IDF, третє на Word2Vec, п'яте на FastText, друге на Doc2Vec і четверте на Sentence-BERT, якщо рахувати кількість строго кращих алгоритмів плюс один та враховувати однакові результати Ward і BIRCH окремо. Це ранжування має сенс лише разом із кількістю кластерів та охопленням: Affinity й щільнісні методи розв'язують завдання з іншою структурою виходу.

Порівняння з MiniBatch особливо показове на Sentence-BERT. Авторський метод поступається за F1, але має вищий ARI; отже, немає одночасної переваги MiniBatch за всіма критеріями. Порівняно зі Spectral у тому самому просторі Parallel K-Means покращує обидві зовнішні метрики, сплачуючи за це більшою затримкою.

Ward є найсильнішою практичною базою для цього малого корпусу, оскільки поєднує високу якість та малий час. Проте його поведінка при зростанні кількості документів потребує окремих вимірювань. Переваги центроїдного й багатостартового підходів на більших даних не підтверджуються й не відхиляються експериментом на 50 текстах.

2.4 Початкові та повторні запуски і стабільність часу

Таблиця 2.8 — Початкове подорожчання векторизації

Подання	Перша серія, мс	Медіана 2–20, мс	Відношення
TF-IDF	3,007	2,561	1,17
Word2Vec	57,112	24,729	2,31
FastText	515,874	531,652	0,97
Doc2Vec	54,895	56,077	0,98
Sentence-BERT	778,893	48,963	15,91

Найбільша зміна спостерігається для Sentence-BERT: 778,893 мс у початковій серії проти 48,963 мс за медіаною повторних. Співвідношення дорівнює 15,91. Це значний ефект першого зафіксованого encode, але він не включає конструктор і завантаження моделі. За наявними даними не можна розкласти цю різницю на окремі внески ініціалізації бібліотек, пристрою та кешів.

Word2Vec у першій серії потребує 57,112 мс проти 24,729 мс надалі, тобто у 2,31 раза більше. Для TF-IDF співвідношення становить 1,17. FastText і Doc2Vec у першій серії виконалися трохи швидше за медіану наступних: відповідно 0,97 та 0,98 від неї. Отже, початкове подорожчання не є однаковим для всіх компонентів.

На етапі кластеризації найвиразніший одиничний ефект отримано для TF-IDF + Ward: 56,546 мс спочатку проти 0,843 мс надалі, приблизно у 67,08 раза. Перший пік відстежуваних виділень становить 645,351 KiB, тоді як повторна медіана — 22,893 KiB. Це узгоджується з додатковою роботою першого виклику, але профіль не визначає її точну причину.

Для Parallel K-Means різниця між початковим і типовим повторним запуском невелика порівняно з цими ефектами. Його K-Means-компоненти вже могли бути прогріті рекомендацією K та попереднім базовим запуском. Тому дані не свідчать про відсутність холодних витрат самого методу; вони показують поведінку в конкретному порядку конвеєра.

Порівняння одного початкового спостереження з 19 повторними дає орієнтир для інтерфейсу користувача: перша обробка може відчутно відрізнятись. Для надійної оцінки справді холодного сценарію потрібні окремі нові процеси з визначеним станом кешів і часовими межами від запуску до готового результату.

Розкид часу та нетипові сповільнення

Таблиця 2.9 — Розкид часу Parallel K-Means у 19 повторних серіях

Подання	Медіана	Середнє	s	Мін.	Макс.
TF-IDF	21,008	21,174	0,749	20,040	23,568
Word2Vec	22,968	53,737	91,847	22,628	316,508
FastText	23,133	23,382	0,895	22,702	26,871
Doc2Vec	22,986	23,193	0,959	22,641	26,878
Sentence-BERT	25,111	25,125	0,428	24,223	25,872

Усі часові значення таблиці 2.9 наведено в мілісекундах; s — вибіркове стандартне відхилення.

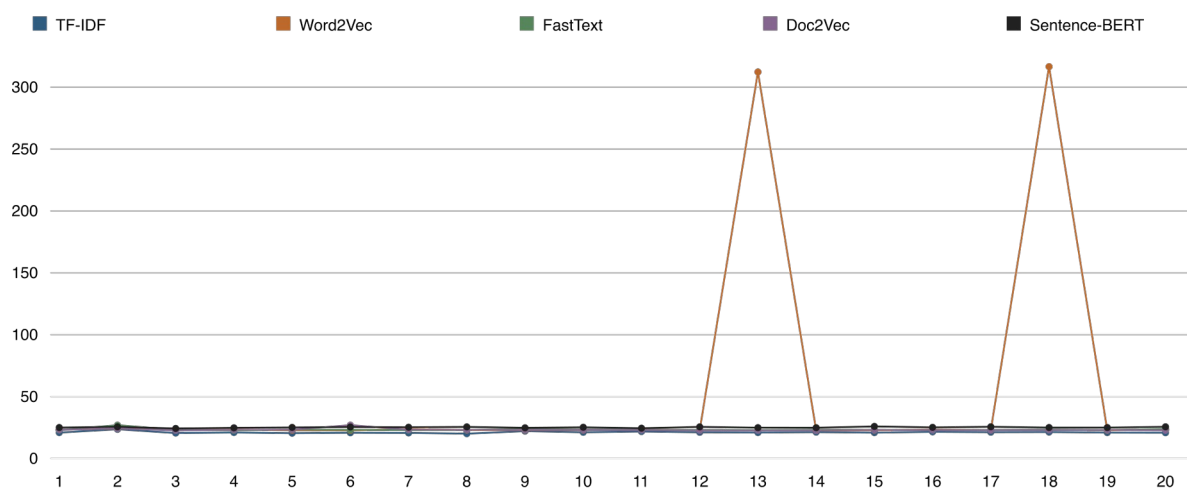


Рисунок 2.3 — Час авторського методу за серіями; вертикальна шкала в мілісекундах

У серіях 13 і 18 Word2Vec + Parallel K-Means виконується за 312,217 та 316,508 мс. Медіана інших і цих повторів разом становить 22,968 мс, середнє — 53,737 мс. Два спостереження не вилучено: вони характеризують можливі затримки цього сеансу. Водночас середнє без медіани завищувало б уявлення про звичайну тривалість приблизно у 2,34 раза.

Якість у цих серіях не змінюється. CPU залишається близьким до інших запусків, але його вікно вимірювання відрізняється від таймера. Без трасування потоків і системних подій причину сповільнень не встановлено. Можливі впливи планувальника чи внутрішніх бібліотек є гіпотезами, а не підтвердженням діагнозом.

Для Sentence-BERT розкид часу авторського методу значно менший: стандартне відхилення близько 0,428 мс при медіані 25,111 мс. Це локальна стабільність у межах одного сеансу. Вона не замінює перевірки різних зерен і корпусів, оскільки повторні метрики якості залишалися незмінними.

2.5 Пам'ять і навантаження процесора на етапах виконання

Таблиця 2.10 — Медіани піків відстежуваних виділень у KiB

Подання	Векторизація	K-Means	Parallel	Ward
TF-IDF	178,9	303,5	1016,9	22,9
Word2Vec	308,3	78,3	294,8	51,9
FastText	781815,9	78,3	287,2	51,9
Doc2Vec	317,8	78,3	278,1	51,9
Sentence-BERT	121,1	261,7	1066,4	162,9

Найбільший зафіксований етапний пік належить формуванню FastText: 781815,945 KiB, або 763,492 MiB за медіаною повторних серій. Для порівняння, пік кластеризації цього подання авторським методом становить 287,222 KiB. Різниця порядків величини показує, що оптимізація лише Parallel K-Means не усуне головної відстежуваної потреби цієї конфігурації в пам'яті.

У Parallel K-Means кожен кандидат створює власну модель, центри, мітки й робочі структури. Вхідна матриця передається зі спільного простору, але внутрішні реалізації можуть створювати додаткові копії. Список результатів зберігає мітки всіх кандидатів до відбору. Тому збільшення піку відносно одного K-Means має зрозумілу структурну основу; конкретний внесок кожної структури окремо не вимірювався.

Найбільші піки кластеризації авторського методу спостерігаються для TF-IDF і Sentence-BERT — приблизно 1016,893 та 1066,382 KiB. Для трьох 100-вимірних подань вони становлять близько 278–295 KiB. Такий розподіл узгоджується з залежністю витрат від форми матриці та робочих масивів, але для точного пояснення потрібні збережені розмірність, тип елементів і профіль алокацій.

Малий пік encode у Sentence-BERT — 121,073 KiB — не є оцінкою розміру моделі. Завантажені ваги створені до запуску tracemalloc, а використання нативної або пристроєвої пам'яті не повністю описується цим показником. Висновок «Sentence-BERT потребує менше RAM за TF-IDF» із наведеної таблиці не впливає.

Практично доцільно розділяти пам'ять довгоживучих об'єктів і тимчасових обчислень. Для інтерактивного застосунку важливо знати базовий обсяг після завантаження моделі, приріст на корпус і максимум під час паралельного пошуку. Поточні дані достатні для порівняння відстежуваних етапних піків, але не для визначення мінімальної RAM, необхідної користувачеві.

Навантаження процесора і дванадцяти ядер

	1	2	3	4	5	6	7	8	9	10	11	12
TF-IDF	94	94	94	94	89	88	100	100	100	100	100	100
Word2Vec	94	94	94	94	93	94	100	100	100	100	100	100
FastText	94	94	94	94	89	88	100	100	100	100	100	100
Doc2Vec	94	94	94	94	94	89	100	100	100	100	100	100
Sentence-BERT	94	94	94	94	89	88	100	100	100	100	100	100

Рисунок 2.4 — Медіани системного навантаження ядер під час Parallel K-Means, %; номери відповідають порядку в експорті

Таблиця 2.11 — Нормований CPU процесу у вікні моніторингу, %

Подання	K-Means	Parallel	Ward	HDBSCAN
TF-IDF	91,63	98,19	47,56	1,52
Word2Vec	91,76	98,26	51,14	1,43
FastText	92,15	97,99	47,52	1,50
Doc2Vec	91,45	98,34	50,76	1,56
Sentence-BERT	91,74	98,16	51,45	1,55

У повторних серіях Parallel K-Means має медіани нормованого CPU близько 98 %. Вони вищі за приблизно 91–92 % базового K-Means. Проте це не вимір прискорення і не доказ рівномірного розподілу восьми кандидатів між ядрами. Час кластеризації авторського методу значно більший, а процесорне вікно охоплює інші межі.

У масивах системного навантаження позиції 7–12 часто мають медіану 100 %, тоді як перші шість — приблизно 88–94 %. За цими індексами не можна визначити тип ядер чи розміщення потоків. Вони не містять ідентифікаторів

фізичної топології та включають сторонню активність. Повні таблиці всіх 45 комбінацій наведено в додатку А.

Дуже різні CPU-показники коротких алгоритмів, зокрема близько 1,5 % у HDBSCAN та понад 90 % у K-Means, не дозволяють безпосередньо ранжувати корисну роботу. Потрібно синхронізувати процесорний час із таймером, відкидати початкові вибірки та зберігати часовий ряд. У цих експериментах CPU є допоміжним описом виконання, а не самостійною мірою енергоефективності.

Ресурси на всіх етапах виконання

Таблиця 2.12 — Повнота наявного ресурсного профілю

Етап	Час і пам'ять	CPU та ядра
Завантаження корпусу	Окремі значення не збережено	Не збережено
Очищення і лематизація	Окремі значення не збережено	Не збережено
Конструктори векторизаторів	Поза профілем; особливо істотно для SBERT	Не збережено
fit_transform	Час і tracemalloc-пік збережено	Вимірюються всередині профайлера, але не експортуються
Підбір K від 2 до 10	Окремо не профілюється	Не збережено
fit_predict	Час і tracemalloc-пік збережено	CPU процесу та системні ядра збережено
Оцінювання метрик	Поза таймером кластеризації	Не збережено
Графіки й дендрограми	Поза таймером кластеризації	Не збережено
Експорт і завершення	Є приблизна рамка за manifest	Не збережено

Таблиця охоплює всі основні етапи фактичного конвеєра. Відсутність окремого вимірювання не означає нульових витрат. Особливо це стосується підбору K, який додає 45 запусків K-Means на кожен серію, та побудови 45 кластерних графіків. Ці роботи можуть помітно впливати на очікування користувача, хоча не входять у порівняльні рядки кластеризаторів.

Для серій 2–20 медіана суми п'яти виміряних векторизацій і 45 кластеризацій становить приблизно 0,945 с. Часова рамка проходження сервісу за назвами каталогів і manifest переважно дорівнює 14–16 с; медіана — 15 с. Для початкової серії відповідні значення становлять 1,766 с і близько 36 с. Різниця не може бути приписана одному етапу без додаткових таймерів.

Серед неврахованих складових є очікування завершення CPU-монітора. Профайлер викликається 50 разів на серію: п'ять векторизацій і 45 кластеризацій. За інтервалу 100 мс його накладні витрати можуть бути істотними відносно мілісекундних операцій. Точне значення цієї складової не збережено. Тому повний час застосунку та суму чистих профільованих викликів у звіті наведено як різні величини.

2.6 Сукупна ефективність і практичний вибір

Таблиця 2.13 — Медіана суми виміряних часу векторизації та кластеризації

Подання	K-Means, мс	Parallel, мс	Ward, мс
TF-IDF	5,312	23,749	3,383
Word2Vec	27,656	47,815	25,522
FastText	535,009	554,785	532,436
Doc2Vec	58,905	79,084	56,888
Sentence-BERT	52,326	74,220	49,863

Показник таблиці 2.13 описує сценарій «одне подання — один алгоритм» у межах двох профільованих операцій. Він не включає завантаження моделі, мовну обробку, підбір K та експорт. Для порівняння повного застосунку потрібна інша межа часу. У серії з дев'ятьма алгоритмами векторизація спільна, тому додавання її дев'ять разів завищувало б сумарні витрати.

У TF-IDF час Parallel K-Means є основною частиною цієї двоетапної суми. У FastText домінує формування подання, тому різниця між кластеризаторами менш помітна в загальній затримці. У Sentence-BERT повторне кодування також дорожче за окрему кластеризацію, проте додаткові старти залишаються відчутними. Отже, оптимізувати потрібно той етап, який визначає витрати конкретного сценарію.

За критеріями «більший F1, більший ARI, менший медіанний час і менший відстежуваний пік кластеризації» Ward перевищує авторський метод на TF-IDF, Word2Vec, FastText і Sentence-BERT. Для Doc2Vec картина складніша: Parallel K-Means покращує F1 та ARI відносно Ward, але витрачає більше ресурсів; Spectral має кращі зовнішні оцінки й менші відповідні витрати. Ці висновки стосуються лише зазначених критеріїв та малого корпусу.

Практична цінність авторського методу в цій роботі полягає у керованому пошуку серед кількох центроїдних розбиттів і помітному покращенні базового

K-Means на семантичних векторах. Його не слід подавати як безумовно найкращий варіант серед усіх доступних алгоритмів. Доцільний сценарій подальшого розвитку — більші корпуси, контрольований бюджет потоків і можливість обирати кількість стартів залежно від допустимої затримки.

Для цього набору даних вибір із найвищим підтвердженням F1 та ARI — Sentence-BERT із Ward або BIRCH. Якщо потрібне саме центроїдне розбиття з пошуком кращого старту, Parallel K-Means із Sentence-BERT є обґрунтованим предметом подальшої оптимізації. Перевагу слід формулювати через конкретний сценарій, а не через універсальний рейтинг.

2.7 Перевірка гіпотез і зіставлення з науковими роботами

Гіпотеза H1 отримала часткову підтримку. Parallel K-Means покращує F1 у трьох поданнях, але одночасно F1 та ARI — лише у Word2Vec і Sentence-BERT. Найбільший ефект у Sentence-BERT узгоджується з припущенням, що багатостартовий пошук корисніший за наявності змістовного простору ознак. Погіршення на TF-IDF і FastText обмежує загальність цього твердження.

Гіпотеза H2 за наявною матрицею не перевірена: немає контрольного послідовного виконання тих самих восьми кандидатів. Повільніший час відносно одного K-Means показує ціну додаткового пошуку, але не визначає ефект його паралельної організації. Гіпотеза H3 підтримується для окремих компонентів, найвиразніше для першого кодування Sentence-BERT; відсутність контрольованого холодного стану залишає причинне пояснення відкритим.

Для H4 виявлено різну структуру витрат пам'яті та часу, особливо домінування векторизації FastText. Повний розподіл CPU між етапами не встановлено, оскільки частина вимірювань не експортується, а системні ядра не атрибутовано процесу. Отже, ця гіпотеза підтверджена лише в межах доступних етапних показників.

Праця Bahmani та співавторів про Scalable K-Means++ розглядає `k-means||` — масштабовану процедуру вибору початкових центрів [8]. Авторський Parallel K-Means у цій роботі організовує незалежні повні запуски звичайного K-Means. Назви пов'язані з паралелізмом, але алгоритмічні механізми різні; результати `k-means||` не є прямим доказом швидкодії реалізованого методу.

Reimers і Gurevych запропонували Sentence-BERT для формування семантичних подань речень [9]. Висока тематична узгодженість цього подання в новій серії відповідає мотиву використання семантичних векторів. Проте

досліджувана багатомовна модель та корпус відрізняються від умов оригінальної публікації, тому пряме числове перевищення чи відтворення її результатів не заявляється.

Українські дослідники Д. В. Ланде та О. О. Рибак аналізують семантичне індексування й кластеризацію документів із кібербезпеки через концепти та їхні зв'язки [10]. Спільним є завдання тематичного впорядкування, але представлення даних відмінне. Порівняння має методологічний характер; кількісне зіставлення потребувало б спільного корпусу, однакової розмітки й критерію якості.

2.8 Власний внесок і напрями подальших досліджень

Власний результат роботи охоплює реалізацію `ParallelKMeansClusterer`, керування кількістю кандидатів і потоків, відбір за `inertia` та включення методу у спільний експериментальний інструмент. Нова серія забезпечила повне порівняння з вісьмома альтернативами у п'яти просторах ознак і дала змогу оцінити повторюваність ресурсів. Це значно ширша емпірична основа, ніж одиничний запуск одного подання.

Для наступного етапу доцільно зафіксувати три контрольні варіанти: один K-Means, вісім послідовних кандидатів і ті самі вісім паралельних. Потрібно експортувати `inertia` кожного кандидата, номер переможця, кількість ітерацій і мітки документів. Це дозволить перевірити, що порівнюється однаковий обсяг пошуку, та відокремити виграш якості від виграшу виконання.

Масштабованість потрібно дослідити на корпусах різного обсягу та складності з окремим варіюванням числа стартів і потоків. Кількість внутрішніх потоків бібліотек слід фіксувати або реєструвати. Повторення з однаковими зернами потрібні для часу; різні набори зерен і незалежні корпуси — для стійкості якості. Просте багаторазове дублювання 50 текстів не замінює змістовного збільшення вибірки.

Ресурсний протокол має зберігати часові межі всіх етапів, CPU user/system до й після кожного, базовий RSS, піковий RSS і часовий ряд системних ядер. Першу неблокувальну вибірку потрібно відкидати в тому самому потоці опитування. Накладні витрати монітора варто виміряти окремо; короткі операції — перевіряти пакетами, не змішуючи очікування потоку з таймером алгоритму.

Холодний сценарій потребує окремого нового процесу для кожного повтору та явно визначеного стану моделі й кешів. Повторний сценарій доцільно перевірити як для нового об'єкта моделі в тому самому процесі, так і для справді повторно використаної моделі. Для користувацького сценарію потрібен повний час від початку обробки до готового звіту з окремим позначенням побудови графіків.

Отримані результати визначають конкретні напрями вдосконалення власного методу: контроль вкладеної багатопотоковості, повторне використання пулу там, де це доречно, збереження діагностики кандидатів і адаптивний бюджет стартів. Ефект кожної зміни має перевірятися новим рівноцінним експериментом; поточний звіт не приписує цим пропозиціям уже досягнутого прискорення.

Висновки

Оцінено авторський Parallel K-Means: вісім незалежних кандидатів, чотири потоки та відбір за мінімальною inertia. Метод інтегровано у спільний конвеєр кластеризації текстів.

Проаналізовано 20 серій від 5 жовтня 2026 року: 45 комбінацій у кожній, 900 записів без помилок. Якість кожної комбінації незмінна з точністю експорту. Ресурси 19 повторних запусків оцінено окремо від початкового.

Найвиразнішу користь авторського методу відносно базового K-Means встановлено для Sentence-BERT: F1 macro зростає з 0,6423 до 0,7024, ARI — з 0,3664 до 0,6072. Для Word2Vec також поліпшуються обидва показники. На TF-IDF і FastText вони погіршуються, а на Doc2Vec змінюються в різних напрямках. Таким чином, ефект додаткових стартів залежить від подання й не гарантує тематичної переваги.

Медіана часу Parallel K-Means становить приблизно 21–25 мс залежно від подання, що у 7,44–8,20 разів перевищує час базового K-Means. Відстежуваний пік пам'яті збільшується у 3,35–4,08 разів. Це ресурсна ціна восьмикандидатного пошуку на 50 документах; чистий ефект паралелізму потребує зіставлення з тими самими вісьмома послідовними стартами.

Початкове кодування Sentence-BERT було у 15,91 разів довшим за медіану повторних викликів. Водночас завантаження моделі не входило до таймера, а перша серія не є контрольованим повністю холодним запуском. Два сповільнення Word2Vec + Parallel K-Means до 312–317 мс збережено в аналізі; вони пояснюють суттєву відмінність середнього від медіани.

Виявлено неповноту етапного профілю ресурсів. CPU збережено лише для кластеризації, а показники дванадцяти ядер описують системну активність. Пам'ять tracemalloc не дорівнює повному RSS. Ці обмеження враховано в інтерпретації та в плані наступних вимірювань.

Найвищі F1 та ARI отримано для Sentence-BERT із Ward і BIRCH. Для авторського методу підтверджено локальне покращення базового K-Means; наступні завдання — перевірка масштабованості та зменшення витрат багатостартового пошуку.

Список використаних джерел

1. Матеріали власних експериментів від 05.10.2026. Каталоги reports/experiments/experiment_20261005_093016 ... experiment_20261005_093602: results_*.json, configuration_*.json, manifest.json. 20 серій, 45 комбінацій у кожній. Послідовність наведено в таблиці 2.1.
2. Text Clustering Research. Вихідний код програми: clustering/parallel_kmeans.py; core/experiment_service.py; vectorizers; preprocessing; metrics. Робоча версія, проаналізована 05.10.2026.
3. Підсистема профілювання програми. benchmark/benchmark_runner.py; benchmark/profiler.py; benchmark/cpu.py; benchmark/memory.py; benchmark/timer.py. Робоча версія, проаналізована 05.10.2026.
4. scikit-learn developers. KMeans. Офіційна документація. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html> (дата звернення: 05.10.2026). Документація пояснює параметри; версії середовища експерименту окремо не зафіксовані.
5. scikit-learn developers. Parallelism and resource management. URL: <https://scikit-learn.org/stable/computing/parallelism.html> (дата звернення: 05.10.2026).
6. psutil developers. Documentation. CPU percentages and per-CPU measurements. URL: <https://psutil.readthedocs.io/stable/index.html> (дата звернення: 05.10.2026).
7. Python Software Foundation. tracemalloc — Trace memory allocations. URL: <https://docs.python.org/uk/3/library/tracemalloc.html> (дата звернення: 05.10.2026).
8. Bahmani B., Moseley B., Vattani A., Kumar R., Vassilvitskii S. Scalable K-Means++. 2012. arXiv:1203.6402. URL: <https://arxiv.org/abs/1203.6402> (дата звернення: 05.10.2026).
9. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP. 2019. P. 3982–3992. DOI: 10.18653/v1/D19-1410. URL: <https://aclanthology.org/D19-1410/> (дата звернення: 05.10.2026).
10. Ланде Д. В., Рибак О. О. Семантичне індексування та кластерний аналіз документів з кібербезпеки. Реєстрація, зберігання і обробка даних. 2024. Т. 26, № 2. DOI: 10.35681/1560-9189.2024.26.2.316711. URL: <https://drsp.ipri.kiev.ua/article/view/316711> (дата звернення: 05.10.2026).
11. Звіт з практики та технічне завдання. Локальні документи «Практика звіт.docx», «Технічне завдання.docx» і матеріали practice_report. Використані як контекст розробленого програмного інструмента.

Додаток А Навантаження ядер для всіх комбінацій

TF-IDF. У таблицях наведено медіани системного навантаження кожного логічного процесора за серіями 2–20, %. Це не частки роботи алгоритму на ядрах. Позиції 1–12 відповідають порядку елементів `cpu_per_core_percent`; топологію фізичних ядер архів не містить.

Таблиця А.1 — TF-IDF, логічні процесори 1–6

Алгоритм	1	2	3	4	5	6
K-Means	25,0	16,7	15,4	9,1	9,1	15,4
Parallel K-Means	94,1	94,1	94,1	94,1	88,9	87,5
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	31,6	30,0	25,0	25,0	25,0	25,0
MiniBatch	94,1	94,1	94,1	93,7	88,9	88,2
BIRCH	94,1	94,1	94,1	93,8	88,2	88,2
Spectral	94,1	94,1	93,7	94,1	88,9	88,2
Affinity	94,1	94,1	94,1	94,1	93,7	88,9
HDBSCAN	33,3	33,3	25,0	23,5	22,2	23,5

Таблиця А.2 — TF-IDF, логічні процесори 7–12

Алгоритм	7	8	9	10	11	12
K-Means	16,7	15,4	16,7	23,1	9,1	16,7
Parallel K-Means	100,0	100,0	100,0	100,0	100,0	100,0
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	28,6	25,0	26,3	30,0	28,6	25,0
MiniBatch	100,0	100,0	100,0	100,0	100,0	100,0
BIRCH	100,0	100,0	100,0	100,0	100,0	100,0
Spectral	100,0	100,0	100,0	100,0	100,0	100,0
Affinity	100,0	100,0	100,0	100,0	100,0	100,0
HDBSCAN	29,4	29,4	25,0	29,4	25,0	27,8

Медіани обчислено окремо для кожної позиції. Вони не є одним одночасним знімком і не повинні підсумовуватися як CPU процесу. Для векторизації, мовної обробки, підбору K, метрик і експорту аналогічні масиви не збережено; відновити їх із цих таблиць неможливо.

Додаток А Продовження Word2Vec

Word2Vec. У таблицях наведено медіани системного навантаження кожного логічного процесора за серіями 2–20, %. Це не частки роботи алгоритму на ядрах. Позиції 1–12 відповідають порядку елементів `cpu_per_core_percent`; топологію фізичних ядер архів не містить.

Таблиця А.3 — Word2Vec, логічні процесори 1–6

Алгоритм	1	2	3	4	5	6
K-Means	23,1	16,7	15,4	9,1	9,1	8,3
Parallel K-Means	94,1	93,8	94,1	93,8	93,3	93,7
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	36,8	33,3	30,0	31,6	30,0	27,8
MiniBatch	94,1	94,1	93,7	93,7	93,7	88,2
BIRCH	94,1	93,8	94,1	93,7	88,9	88,2
Spectral	94,1	94,1	93,7	93,7	88,2	87,5
Affinity	100,0	94,1	94,1	94,1	93,7	87,5
HDBSCAN	35,3	31,2	26,7	29,4	25,0	23,5

Таблиця А.4 — Word2Vec, логічні процесори 7–12

Алгоритм	7	8	9	10	11	12
K-Means	16,7	9,1	16,7	16,7	16,7	9,1
Parallel K-Means	100,0	100,0	100,0	100,0	100,0	100,0
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	30,0	30,0	30,0	30,0	30,0	30,0
MiniBatch	100,0	100,0	100,0	100,0	100,0	100,0
BIRCH	100,0	100,0	100,0	100,0	100,0	100,0
Spectral	100,0	100,0	100,0	100,0	100,0	100,0
Affinity	100,0	100,0	100,0	100,0	100,0	100,0
HDBSCAN	29,4	31,2	35,3	31,3	29,4	29,4

Медіани обчислено окремо для кожної позиції. Вони не є одним одночасним знімком і не повинні підсумовуватися як CPU процесу. Для векторизації, мовної обробки, підбору K, метрик і експорту аналогічні масиви не збережено; відновити їх із цих таблиць неможливо.

Додаток А Продовження FastText

FastText. У таблицях наведено медіани системного навантаження кожного логічного процесора за серіями 2–20, %. Це не частки роботи алгоритму на ядрах. Позиції 1–12 відповідають порядку елементів `cpu_per_core_percent`; топологію фізичних ядер архів не містить.

Таблиця А.5 — FastText, логічні процесори 1–6

Алгоритм	1	2	3	4	5	6
K-Means	16,7	15,4	15,4	15,4	9,1	9,1
Parallel K-Means	94,4	94,1	94,1	94,1	88,9	88,2
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	35,0	30,0	28,6	25,0	25,0	25,0
MiniBatch	94,1	94,1	93,7	93,8	88,9	88,2
BIRCH	94,1	94,1	94,1	94,1	88,9	88,2
Spectral	94,4	94,1	94,1	93,7	88,9	88,2
Affinity	94,4	94,4	94,1	94,1	94,1	88,9
HDBSCAN	31,3	27,8	23,5	22,2	18,7	18,8

Таблиця А.6 — FastText, логічні процесори 7–12

Алгоритм	7	8	9	10	11	12
K-Means	9,1	9,1	9,1	9,1	16,7	9,1
Parallel K-Means	100,0	100,0	100,0	100,0	100,0	100,0
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	28,6	27,3	26,3	26,3	27,3	25,0
MiniBatch	100,0	100,0	100,0	100,0	100,0	100,0
BIRCH	100,0	100,0	100,0	100,0	100,0	100,0
Spectral	100,0	100,0	100,0	100,0	100,0	100,0
Affinity	100,0	100,0	100,0	100,0	100,0	100,0
HDBSCAN	27,8	27,8	27,8	27,8	27,8	23,5

Медіани обчислено окремо для кожної позиції. Вони не є одним одночасним знімком і не повинні підсумовуватися як CPU процесу. Для векторизації, мовної обробки, підбору K, метрик і експорту аналогічні масиви не збережено; відновити їх із цих таблиць неможливо.

Додаток А Продовження Doc2Vec

Doc2Vec. У таблицях наведено медіани системного навантаження кожного логічного процесора за серіями 2–20, %. Це не частки роботи алгоритму на ядрах. Позиції 1–12 відповідають порядку елементів `cpu_per_core_percent`; топологію фізичних ядер архів не містить.

Таблиця А.7 — Doc2Vec, логічні процесори 1–6

Алгоритм	1	2	3	4	5	6
K-Means	25,0	10,0	16,7	9,1	9,1	8,3
Parallel K-Means	94,1	94,1	93,7	94,1	93,7	88,9
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	35,0	33,3	30,0	28,6	28,6	26,3
MiniBatch	94,1	94,1	94,1	93,7	88,9	88,2
BIRCH	94,1	94,1	93,7	93,7	88,9	88,2
Spectral	94,1	94,1	94,1	93,7	88,2	88,9
Affinity	94,1	94,1	94,4	94,1	93,7	88,2
HDBSCAN	35,3	31,2	27,8	25,0	23,5	23,5

Таблиця А.8 — Doc2Vec, логічні процесори 7–12

Алгоритм	7	8	9	10	11	12
K-Means	16,7	9,1	16,7	16,7	18,2	15,4
Parallel K-Means	100,0	100,0	100,0	100,0	100,0	100,0
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	30,0	28,6	30,0	30,0	30,0	28,6
MiniBatch	100,0	100,0	100,0	100,0	100,0	100,0
BIRCH	100,0	100,0	100,0	100,0	100,0	100,0
Spectral	100,0	100,0	100,0	100,0	100,0	100,0
Affinity	100,0	100,0	100,0	100,0	100,0	100,0
HDBSCAN	29,4	33,3	29,4	29,4	29,4	26,7

Медіани обчислено окремо для кожної позиції. Вони не є одним одночасним знімком і не повинні підсумовуватися як CPU процесу. Для векторизації, мовної обробки, підбору K, метрик і експорту аналогічні масиви не збережено; відновити їх із цих таблиць неможливо.

Додаток А Продовження Sentence-BERT

Sentence-BERT. У таблицях наведено медіани системного навантаження кожного логічного процесора за серіями 2–20, %. Це не частки роботи алгоритму на ядрах. Позиції 1–12 відповідають порядку елементів `cpu_per_core_percent`; топологію фізичних ядер архів не містить.

Таблиця А.9 — Sentence-BERT, логічні процесори 1–6

Алгоритм	1	2	3	4	5	6
K-Means	27,3	16,7	9,1	9,1	15,4	9,1
Parallel K-Means	94,1	94,1	94,1	94,1	88,9	88,2
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	35,0	30,0	30,0	26,3	26,3	26,3
MiniBatch	94,1	94,1	94,1	93,7	88,9	88,2
BIRCH	94,4	94,1	94,1	93,7	93,7	88,2
Spectral	94,1	94,1	94,1	94,1	88,9	88,2
Affinity	94,1	94,1	94,1	94,1	94,1	88,2
HDBSCAN	31,3	29,4	29,4	23,5	23,5	18,8

Таблиця А.10 — Sentence-BERT, логічні процесори 7–12

Алгоритм	7	8	9	10	11	12
K-Means	16,7	15,4	16,7	15,4	9,1	15,4
Parallel K-Means	100,0	100,0	100,0	100,0	100,0	100,0
Ward	100,0	100,0	100,0	100,0	100,0	100,0
DBSCAN	30,0	30,0	28,6	30,0	28,6	28,6
MiniBatch	100,0	100,0	100,0	100,0	100,0	100,0
BIRCH	100,0	100,0	100,0	100,0	100,0	100,0
Spectral	100,0	100,0	100,0	100,0	100,0	100,0
Affinity	100,0	100,0	100,0	100,0	100,0	100,0
HDBSCAN	29,4	29,4	27,8	29,4	27,8	25,0

Медіани обчислено окремо для кожної позиції. Вони не є одним одночасним знімком і не повинні підсумовуватися як CPU процесу. Для векторизації, мовної обробки, підбору K, метрик і експорту аналогічні масиви не збережено; відновити їх із цих таблиць неможливо.

Додаток Б Розподіли часу всіх комбінацій

Статистики обчислено за 19 повторними серіями для кожної пари. Усі значення наведено в мілісекундах; s — вибіркове стандартне відхилення, Q1–Q3 — міжквартильний інтервал. Ці таблиці доповнюють медіани основного розділу та дозволяють перевірити стабільність кожного методу, зокрема вплив поодиноких затримок.

Таблиця Б.1 — TF-IDF, час кластеризації у повторному режимі

Алгоритм	Середнє	s	Q1–Q3	Мін.	Макс.
K-Means	2,743	0,108	2,685–2,795	2,519	2,961
Parallel K-Means	21,174	0,749	20,812–21,332	20,040	23,568
Ward	0,843	0,030	0,836–0,851	0,756	0,910
DBSCAN	1,043	0,115	0,941–1,132	0,910	1,256
MiniBatch	6,681	1,140	5,872–6,824	5,628	10,586
BIRCH	2,192	0,033	2,171–2,206	2,148	2,279
Spectral	12,766	0,453	12,575–12,738	12,486	14,467
Affinity	8,439	0,057	8,410–8,461	8,323	8,597
HDBSCAN	2,103	0,070	2,072–2,103	2,048	2,355

Таблиця Б.2 — Word2Vec, час кластеризації у повторному режимі

Алгоритм	Середнє	s	Q1–Q3	Мін.	Макс.
K-Means	2,974	0,266	2,758–3,188	2,579	3,398
Parallel K-Means	53,737	91,847	22,800–23,368	22,628	316,508
Ward	0,786	0,012	0,777–0,794	0,770	0,809
DBSCAN	1,240	0,192	1,106–1,424	1,063	1,680
MiniBatch	5,429	1,263	4,703–5,536	4,615	8,873
BIRCH	1,431	0,043	1,411–1,436	1,377	1,551
Spectral	13,897	1,857	13,152–13,487	13,050	20,902
Affinity	4,253	0,036	4,233–4,267	4,201	4,358
HDBSCAN	1,780	0,059	1,757–1,787	1,729	1,980

Таблиця Б.3 — FastText, час кластеризації у повторному режимі

Алгоритм	Середнє	s	Q1–Q3	Мін.	Макс.
K-Means	2,958	0,386	2,623–3,244	2,535	3,640
Parallel K-Means	23,382	0,895	23,027–23,298	22,702	26,871
Ward	0,829	0,191	0,774–0,794	0,757	1,613

Алгоритм	Середнє	s	Q1–Q3	Мін.	Макс.
DBSCAN	1,244	0,185	1,074–1,438	1,027	1,550
MiniBatch	4,993	0,980	4,369–5,411	4,309	7,298
BIRCH	1,443	0,114	1,398–1,430	1,386	1,899
Spectral	13,138	1,266	12,469–13,417	12,310	17,434
Affinity	4,288	0,036	4,268–4,300	4,246	4,399
HDBSCAN	1,739	0,087	1,692–1,728	1,671	1,962

Таблиця Б.4 — Doc2Vec, час кластеризації у повторному режимі

Алгоритм	Середнє	s	Q1–Q3	Мін.	Макс.
K-Means	2,820	0,097	2,746–2,876	2,690	3,020
Parallel K-Means	23,193	0,959	22,789–23,060	22,641	26,878
Ward	0,806	0,077	0,783–0,795	0,771	1,120
DBSCAN	1,304	0,189	1,086–1,440	1,040	1,562
MiniBatch	5,838	1,194	5,179–5,919	5,031	9,457
BIRCH	1,488	0,291	1,413–1,432	1,372	2,685
Spectral	13,863	1,213	13,506–13,602	13,454	18,812
Affinity	5,138	0,061	5,103–5,142	5,087	5,351
HDBSCAN	1,855	0,031	1,839–1,864	1,816	1,959

Таблиця Б.5 — Sentence-BERT, час кластеризації у повторному режимі

Алгоритм	Середнє	s	Q1–Q3	Мін.	Макс.
K-Means	3,379	0,142	3,294–3,436	3,120	3,805
Parallel K-Means	25,125	0,428	24,860–25,507	24,223	25,872
Ward	0,886	0,018	0,877–0,895	0,850	0,932
DBSCAN	1,539	0,231	1,355–1,681	1,129	1,977
MiniBatch	5,682	0,804	5,311–5,520	5,260	8,449
BIRCH	2,450	0,086	2,380–2,535	2,368	2,604
Spectral	13,661	0,793	13,327–13,465	13,277	16,020
Affinity	4,803	0,030	4,782–4,822	4,745	4,855
HDBSCAN	2,515	0,059	2,481–2,513	2,460	2,673